

RecallGuard – Final Verification Report

Conversational Memory Intelligence System

1. Verification Objective

This document records the final verification status of the RecallGuard conversational memory system after completing the core implementation, testing, memory lifecycle, reflection, and local LLM integration work.

2. Verified Components

Component	Verification	Status
Memory extraction	Preferences, goals, facts and dislikes detected	PASS
Admission policy	Importance, confidence and store decision evaluated	PASS
Memory storage	Memories persisted in ChromaDB	PASS
Duplicate detection	Semantic similarity prevents repeated memories	PASS
Retrieval	Relevant memories returned for user queries	PASS
Ranking	Retrieved memories ranked before context construction	PASS
Context builder	Relevant memories included in bounded prompt	PASS
Chat pipeline	Message → memory → retrieval → context → response	PASS
Reflection	Summary and lifecycle maintenance executed	PASS
Deletion	Soft deletion supported through memory endpoint	PASS
Authentication	API-key authentication enforced on protected endpoints	PASS
User isolation	Memory queries filtered by user_id	PASS
Local LLM	Free/local model connected to chat pipeline	PASS

3. End-to-End Verification

Pipeline verified: User message → extraction → admission → duplicate detection → memory storage → semantic retrieval → ranking → context construction → local LLM response.

The system successfully demonstrated memory-aware responses such as retrieving a previously stored coffee preference and using relevant memories in the generated response. It also demonstrated preference updates, where newer information such as a change from coffee to tea can be represented as a newer memory.

4. Reflection Verification

The /reflection endpoint was tested successfully. A representative result reported 6 active memories with 2 conversation memories, 1 fact, 1 goal, 2 preferences and 0 tasks. The maintenance result reported 0 expired memories, 0 duplicates removed and 0 low-confidence memories removed for that run.

5. Retrieval Verification

The retrieval endpoint was verified using semantic queries. Relevant memories such as coffee/tea preferences, Python-learning facts and the AI Engineer goal were retrieved and passed to the context builder.

6. Security Verification

Protected endpoints use API-key authentication. User-level filtering is applied when accessing memories. Input validation, duplicate protection and soft deletion are implemented. The security report separately

documents production hardening items such as HTTPS, rate limiting, secure secret management and disabling Flask debug mode in production.

7. Evaluation Artifacts

The verification package includes a test plan, individual test records, retrieval results, an evaluation dataset, and a security report. These artifacts provide evidence for the major memory-system components and their expected behavior.

8. Final Status

FINAL VERIFICATION: PASS

The core RecallGuard memory pipeline is implemented and verified. The project demonstrates memory extraction, admission, persistent storage, semantic duplicate detection, retrieval, context construction, reflection/lifecycle maintenance, authentication, and local LLM-based conversation.

9. Remaining Production Improvements

- Move API keys and secrets out of source code.
- Run behind a production WSGI server instead of Flask's development server.
- Enable HTTPS/TLS for deployed environments.
- Add rate limiting and stronger API-key rotation.
- Add automated regression tests for the complete pipeline.